

Four-Member Task Workflow

Project: Sports Registration · Class: [Class/Semester] · Coordinator: [Name]

Team: Member 1 [Name] · Member 2 [Name] · Member 3 [Name] · Member 4 [Name]

Work sequence

Phase 0 — Repository setup (Member 1 coordinates)

- [] Create the GitHub repository and add the other three members as collaborators.
- [] Push the project baseline; check `.gitignore` excludes `.env`, `instance/`, `*.db`, `.venv/` and `venv/`.
- [] Each person confirms their own Git identity and can clone/pull the repo.

Gate: all four can see the same repository and the project setup instructions.

Phase 1 — Agree on the interface contract (Members 1 and 2)

- [] Member 1 confirms route names, form fields and template variables.
- [] Member 2 confirms which templates use those names and will keep them unchanged.
- [] Member 3 copies the required end-to-end cases into `MEMBER_3_TESTING.md`.
- [] Member 4 confirms the seed records and demo steps match the current app.

Gate: no team member renames a route or form field without notifying the others.

Phase 2 — Small parallel tasks

- [] **Member 1:** maintain/fix Flask routes, models, validation, approvals and CSV behavior; store the event background override, resolve Auto by sport, expose the next upcoming event, and keep existing SQLite data through the targeted schema update.
- [] **Member 2:** deliver the sports-first HTML/Jinja and CSS: matchday hero, next-event scoreboard, sport accent themes, and clear admin preset selector. The detailed UI code chunks/checks are in `MEMBER_2_UI_UX.md`.
- [] **Member 3:** test Auto mapping, admin override on create/edit, next-event selection, and the public/admin flows; record defects/retest results.
- [] **Member 4:** verify seed data, setup instructions and demo runbook; demonstrate an actual theme override and prepare genuine screenshots.

Each member uses one task branch for related work. After each genuine work period, test the slice, make a factual commit if code changed, and push the branch so the team can review progress. See `GITHUB_WORKFLOW.md`; do not invent/backdate activity.

Phase 3 — Review and integration

- [] Member 1 reviews the UI PR for route/form compatibility.
- [] Member 2 reviews the visible form and dashboard flow after backend changes.
- [] Member 3 runs `python -m unittest discover -s tests -v` after merges and completes the manual checklist.
- [] Member 4 follows `DEMO_RUNBOOK.md` from a clean setup and checks screenshots.
- [] Merge reviewed pull requests into `main`; all members pull the latest version.

Phase 4 — Final demonstration

Run the complete flow:

```
Admin adds event
→ event page appears
→ student submits participant/volunteer form
→ registration is stored in SQLite as Pending
→ admin views the student record
→ admin approves or rejects
→ status statistics update
→ admin exports CSV
```

Definition of done

- ☐ Event creation, event page and both registration types work.
- ☐ Deadline, duplicate and configured capacity checks behave as expected.
- ☐ Admin can view, approve/reject, see counts and export CSV.
- ☐ Four roles have small, specific, verifiable tasks.
- ☐ Tests and demo were run; real results are recorded.
- ☐ No fake/empty commits or invented contributions are included.